

Lesson 9.3 — Demo Runbook (Laravel-only, з конкретним кодом)

Цей файл — єдина інструкція для мітингу: **слайди -> цей runbook -> назад до слайдів**.

Під час демо `lesson_9_3_practice.md` не відкриваємо.

0) Відповідність презентації

У `lesson_9_3_presentation.pdf` практичні переходи позначені як «Слайд → demo». Покриваємо їх у runbook так:

- **Слайд 9а — HTTP-кеш для списків і довідників** → Блок А
- **Слайд 11а — обсяг відповіді (DTO/Resources, поля, розмір JSON)** → Блок В
- **Слайд 13а — rate limiting (`throttle`)** → Блок С
- **Слайд 15а — асинхронний експорт через job (202 Accepted)** → Блок D
- **Фінальний чек-лист production-ready API** → Блок Е (використовується як підсумок)

Важливо: поточний робочий код проекту (`AccountController`, `PaymentController`, `PaymentService`, `PaymentRepository`, тести) **не переписуємо**. Усі приклади нижче — точкові «навчальні» вставки, які показуємо під час мітингу.

1) Pre-flight (до початку лекції)

1. Перевірити контейнери:

- `docker compose ps`

2. Перевірити маршрути:

- `docker compose exec -T php php artisan route:list --path=api/v1`

3. Перевірити тести на SQLite:

- `docker compose exec -T php php artisan test tests/Feature/Api/V1`

4. Перевірити, що Telescope працює (встановлений ще в 9.2):

- `http://localhost/telescope/requests`

Якщо Telescope/Debugbar з якоїсь причини відсутні — використай блоки встановлення з `docs/9.2/lesson_9_2_demo_runbook.md`, секція «Встановлення Telescope / Debugbar».

Базовий URL проекту при `docker compose` — `http://localhost`.

2) Блок А — HTTP-кеш (перехід зі слайда 9а)

2.1 Що говорити

- HTTP-кеш живе **на клієнті/проксі**, а не в нашому коді.
- Доповнює серверний Redis-кеш (із 9.2), а не замінює його.
- Для фінансових даних — **private** і **короткий max-age** (або **no-store**).
- Для публічних довідників (валюти, тарифи) — **public** + довгий **max-age**.

2.2 Навчальна переробка **AccountController::payments** (показуємо як приклад)

Мета: для списку платежів акаунта віддавати **Cache-Control: private, max-age=15** і **ETag**, щоб повторні запити UI отримували **304**.

```
public function payments(Account $account, AccountPaymentsIndexRequest
$request): \Illuminate\Http\JsonResponse
{
    $query = AccountPaymentsQueryDTO::fromValidated($request-
>validated());
    $payments = $this->paymentService->listPaymentsForAccount((int)
$account->id, $query->getPerPage());

    $payload = PaymentResource::collection($payments)->response()-
>getData(true);
    $etag = '' . sha1((string) json_encode($payload)) . '';

    if ($request->headers->get('If-None-Match') === $etag) {
        return response()->json(null, 304)
            ->header('ETag', $etag)
            ->header('Cache-Control', 'private, max-age=15');
    }

    return response()->json($payload)
        ->header('ETag', $etag)
        ->header('Cache-Control', 'private, max-age=15');
}
```

Що в цьому методі нового саме за темою 9.3 (HTTP-кеш):

Базова частина методу (DTO, сервіс, **PaymentResource**) уже була зроблена в 9.1 — її не розбираємо, вона тут лише щоб отримати тіло відповіді. Урок 9.3 додає три речі поверх неї.

1. Рахуємо **ETag** — «відбиток» поточної відповіді.

```
$payload = PaymentResource::collection($payments)->response()-
>getData(true);
$etag = '' . sha1((string) json_encode($payload)) . '';
```

Беремо фінальний JSON, рахуємо від нього SHA-1 і беремо в лапки (так вимагає HTTP-стандарт: **ETag: "abc..."**). Ідея проста: однакові дані → однаковий ETag, змінився хоча б один платіж → ETag інший. Це і є «версія» відповіді, яку клієнт зможе порівняти. 2. **Обробляємо умовний запит If-None-Match → віддаємо 304.**

```

if ($request->headers->get('If-None-Match') === $etag) {
    return response()->json(null, 304)
        ->header('ETag', $etag)
        ->header('Cache-Control', 'private, max-age=15');
}

```

Клієнт при повторному GET надсилає попередній ETag у заголовку **If-None-Match**. Якщо збіглося — дані не змінювалися, і ми відповідаємо **304 Not Modified без тіла**: клієнт візьме свою попередню копію. Це і є вигравш HTTP-кешу — ми не серіалізуємо JSON і не женемо його мережею. 3. **На повній відповіді ставимо ETag і Cache-Control**.

```

return response()->json($payload)
    ->header('ETag', $etag)
    ->header('Cache-Control', 'private, max-age=15');

```

- **ETag** клієнт збереже і надішле наступного разу — без нього весь механізм 304 не працює.
- **Cache-Control: private, max-age=15** — політика кешу:
 - **private** — кешувати можна **лише на самому клієнті** (браузер, мобільний застосунок). Проксі/CDN — не можна, бо список платежів **персональний**.
 - **max-age=15** — 15 секунд клієнт може використовувати свою копію взагалі не звертаючись до сервера. Для типового UX (натиснув «назад» / перемкнув вкладку) цього достатньо, при цьому суми не виглядають «застарілими».

Коротко: ETag дає «дешеві 304» для повторних запитів, а **Cache-Control** каже клієнту, **кому і на скільки** дозволено кешувати.

У проєкті вже підготовлений порожній метод-заглушка:

- **AccountController::paymentsCached()** (поруч із **payments()**),
- маршрут **GET /api/v1/accounts/{account}/payments-cached** → **accounts.payments.cached**.

Зараз він просто повертає **{"data": [], "message": "TODO: ..."}**. На демо я відкриваю файл і **вживу набираю** код за схемою вище (DTO → service → ETag → 304 / 200 + **Cache-Control**). Старий **payments()** залишається «як було», і студенти одразу бачать різницю: один віддає звичайний 200, інший — 200 з **ETag/Cache-Control** і потім 304 на повторі.

2.3 Навчальний **CurrencyController** для публічного кешу (за слайдом)

Для довідника валют приклад із **public, max-age=3600**:

```

public function index(): \Illuminate\Http\JsonResponse
{
    $currencies = ['USD', 'EUR', 'UAH'];

    return response()

```

```
->json(['data' => $currencies])
->header('Cache-Control', 'public, max-age=3600');
}
```

Що в цьому методі нового саме за темою 9.3 (HTTP-кеш):

Сама видача довідника тривіальна (масив або `CurrencyService::getAll()` із 9.2). Урок 9.3 додає один заголовок — і саме його й обговорюємо.

```
->header('Cache-Control', 'public, max-age=3600');
```

- **public** — кешувати дозволено **де завгодно**: браузер, корпоративний проксі, CDN. Це безпечно, бо довідник валют **не персоналізований** і не містить чутливих даних. Порівняй із прикладом вище, де для платежів ми ставимо **private** саме через персональність.
- **max-age=3600** — година. До закінчення цього строку клієнти і проксі навіть **не будуть** звертатися до нашого сервера за цим ресурсом — тобто ми виносимо навантаження за межі застосунку взагалі.

І ще одна важлива думка на цьому прикладі: **HTTP-кеш і серверний Redis-кеш — це різні шари**. Якщо всередині методу стоїть `CurrencyService::getAll()` із `Cache::remember(...)` із 9.2, ми отримуємо подвійний виграш:

- HTTP-кеш гасить запити **до** Laravel (клієнт бере зі своєї копії);
- Redis-кеш гасить роботу **всередині** Laravel, коли запит таки дійшов.

Контраст двох прикладів запам'ятати просто:

- список платежів акаунта — **private, max-age=15** (особисте, ненадовго);
- довідник валют — **public, max-age=3600** (загальне, надовго).

Вибір **public/private** і **max-age** — це і є основне практичне рішення з HTTP-кешу для кожного endpoint-a.

У проєкті вже підготовлено порожній `CurrencyController`:

- клас `App\Http\Controllers\Api\V1\CurrencyController` з порожнім методом `index()`,
- маршрут `GET /api/v1/currencies` → `currencies.index`.

На демо відкриваю файл і вживу дописую повернення списку валют + заголовок `Cache-Control: public, max-age=3600`.

2.4 Live-демо (порівняння `payments` vs `payments-cached` vs `currencies`)

Кроки під час мітингу:

1. Створити акаунт (якщо ще немає id):

- `curl -s -X POST http://localhost/api/v1/accounts -H "Content-Type: application/json" -d '{"balance":"5000.00"}'`

2. Базовий endpoint без HTTP-кешу — подивитися заголовки:

- `curl -i "http://localhost/api/v1/accounts/ID/payments?per_page=20"`
- У відповіді **немає** `ETag` і `Cache-Control` — кожен клієнтський запит іде «повністю».

3. Endpoint із клієнтським HTTP-кешем (після того, як я дописав `paymentsCached`):

- перший запит — отримуємо тіло + `ETag`:
 - `curl -i "http://localhost/api/v1/accounts/ID/payments-cached?per_page=20"`
- копіюємо значення із заголовка `ETag`: "...";
- повторний запит з `If-None-Match` — отримуємо `304 Not Modified` без тіла:
 - `curl -i -H 'If-None-Match: "СКОПІЙОВАНИЙ_ETAG"' "http://localhost/api/v1/accounts/ID/payments-cached?per_page=20"`
- дивимось заголовок `Cache-Control: private, max-age=15`.

4. Публічний кеш для довідника валют (після `CurrencyController::index`):

- `curl -i "http://localhost/api/v1/currencies"`
- у відповіді `Cache-Control: public, max-age=3600` — клієнт/проксі/CDN можуть кешувати на годину, до нашого сервера повторно не підуть.

5. Підсвітити різницю: один і той самий шаблон (`Cache-Control` + опційно `ETag`), але різні політики під різні дані:

- персональний список → `private, max-age=15 + ETag`;
- публічний довідник → `public, max-age=3600`.

2.5 Що підкреслити

- HTTP-кеш — це **безкоштовний виграш** на повторних GET.
- Для *real-time* грошових endpoint-ів (баланс перед списанням) — `no-store`.
- `ETag` + `If-None-Match` дають 304 без тіла відповіді.

3) Блок В — обсяг відповіді: DTO/Resources (перехід зі слайда 11a)

3.1 Що говорити

- Менше полів → менше байтів → швидша серіалізація і мережа.
- Resource/DTO — це і архітектура, і перформанс одночасно.
- Списки особливо чутливі: кожне зайве поле множиться на розмір сторінки.

3.2 Що вже є у проєкті (готово до запуску)

Для прямого порівняння «жирний vs чистий» у проєкті вже підготовлено:

- «Чистий» варіант — GET /api/v1/accounts/{account}/payments через App\Http\Resources\PaymentResource (мінімальний набір полів: id, account_id, amount, currency, description, status, created_at).
- «Жирний» варіант — GET /api/v1/accounts/{account}/payments-fat через App\Http\Resources\PaymentFatResource (той самий сервіс/репозиторій, але Resource навмисно роздутий: amount_minor_units, commission, gateway_payment_id, gateway_raw_response (вкладений об'єкт із trace), internal_note, audit.host/environment, created_at_iso, created_at_unix, updated_at, updated_at_iso, description_length, status_label, is_processed тощо).

Тобто обидва endpoint-и повертають **один і той самий набір платежів одного й того ж акаунта**, відрізняється лише Resource — це дає чисте порівняння за обсягом відповіді та часом серіалізації.

3.3 Що показати на демо (живий замір у Telescope)

1. Відкрити <http://localhost/telescope/requests>.
2. (Якщо даних мало) згенерувати платежі фабрикою — це стане в пригоді і для блока B, і для блока з індексами:
 - `docker compose exec -T php php artisan tinker --execute="\App\Models\Payment::factory()->count(2000)->create(['account_id' => ID, 'status' => 'processed', 'currency' => 'USD']);"`
3. Зробити по 3 запити кожного варіанта, щоб Telescope їх зафіксував:
 - `for i in {1..3}; do curl -s "http://localhost/api/v1/accounts/ID/payments?per_page=50" > /dev/null; done`
 - `for i in {1..3}; do curl -s "http://localhost/api/v1/accounts/ID/payments-fat?per_page=50" > /dev/null; done`
4. У Telescope (вкладка **Requests**) відкрити обидва варіанти по черзі й зафіксувати на дошці:
 - **Duration** (загальний час запиту);
 - **Memory**;
 - розмір тіла відповіді (видно в **Response** або за **Content-Length**);
 - **Queries** (кількість запитів і час) — має бути однаковим, бо репозиторій той самий; різниця йде саме на серіалізації та мережі.

3.4 Що підкреслити студентам

- Resource — це і архітектура, і перформанс. Одна й та сама вибірка з БД може стати в **рази** важчою в мережі лише через «жирну» відповідь.
 - На великих **per_page** різниця множиться: кожне зайве поле × **per_page** рядків × частота запитів = реальне навантаження.
 - «Жирні» поля на кшталт **gateway_raw_response** або **internal_note** — це ще й **витік внутрішніх деталей**. Контроль набору полів у Resource = security by design.
 - Правило: якщо поле не потрібне жодному поточному клієнту — його не повинно бути у відповіді. Під окремий кейс робить **окремий** Resource (наприклад, **PaymentDetailsResource** для сторінки платежу), а не «все й одразу» в списку.
-

4) Блок C — rate limiting (перехід зі слайда 13а)

4.1 Що говорити

- `throttle:N,M` — N запитів за M хвилин на користувача/токен/IP.
- Різні endpoint-и — різні ліміти (звичайні API і «важкі» звіти).
- 429 — це **нормальна** відповідь, фронт має показати повідомлення і `retry/backoff`.

4.2 Що таке `throttle` і чи потрібно його створювати

Своє middleware створювати не потрібно. `throttle` — це вбудований alias на клас `Illuminate\Routing\Middleware\ThrottleRequests` (в API-стеку Laravel 11/12 — `ThrottleRequestsWithRedis`). Він зареєстрований фреймворком і доступний «з коробки»: достатньо просто навісити його на маршрути.

Під капотом він:

- визначає ключ клієнта (для гостя — IP, для авторизованого — id користувача або токена Sanctum);
- інкрементує лічильник у кеші (`config/cache.php`, зазвичай Redis у цьому проєкті);
- порівнює з лімітом, при перевищенні повертає `429 Too Many Requests + Retry-After`;
- на будь-якій відповіді ставить `X-RateLimit-Limit` і `X-RateLimit-Remaining`.

Тобто `throttle` сам по собі — **не сервіс**, не конфіг, не ваш файл; це middleware Laravel. Створювати нові класи під нього не потрібно.

4.3 Спосіб 1 — `throttle:N,M` прямо на маршрутах (мінімум коду)

Найпростіший спосіб — вказати ліміт як параметри alias прямо в `routes/api.php`:

```
Route::prefix('v1')->group(function () {
    Route::middleware('throttle:60,1')->group(function () {
        Route::get('accounts/{account}/payments',
            [AccountController::class, 'payments'])
            ->name('accounts.payments.index');
        Route::apiResource('payments', PaymentController::class)-
>except(['update']);
        Route::apiResource('accounts', AccountController::class)-
>except(['update']);
    });

    Route::middleware('throttle:5,1')->group(function () {
        Route::post('reports/account-statement',
            [\App\Http\Controllers\Api\V1\ReportsController::class,
            'generateAccountStatement'])
            ->name('reports.account-statement');
    });
});
```

Що означають параметри:

- `throttle:60,1` → 60 запитів за 1 хвилину;
- `throttle:5,1` → 5 запитів за 1 хвилину (для важких звітів).

Ключ групування вибирається автоматично: для гостя — IP, для авторизованого — `Auth::id()`.

4.4 Спосіб 2 — іменований `RateLimiter` (рекомендується для production)

Коли потрібна більша гнучкість (різні ключі, різні ліміти для гостя/юзера, кастомна відповідь), Laravel пропонує реєструвати **іменовані limiter-и** через фасад `RateLimiter`. Це **вбудований механізм фреймворка**, своїх класів писати не треба.

В `app/Providers/AppServiceProvider.php` (метод `boot`):

```
use Illuminate\Cache\RateLimiting\Limit;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\RateLimiter;

public function boot(): void
{
    RateLimiter::for('api', function (Request $request): Limit {
        return $request->user()
            ? Limit::perMinute(120)->by((string) $request->user()->id)
            : Limit::perMinute(30)->by((string) $request->ip());
    });

    RateLimiter::for('reports', function (Request $request): Limit {
        return Limit::perMinute(5)->by((string) ($request->user()?->id ??
$request->ip()));
    });
}
```

Тоді в `routes/api.php` достатньо послатися на ім'я:

```
Route::middleware('throttle:api')->group(function () {
    Route::get('accounts/{account}/payments', [AccountController::class,
'payments']);
});

Route::middleware('throttle:reports')->group(function () {
    Route::post('reports/account-statement', [ReportsController::class,
'generateAccountStatement']);
});
```

Що це дає:

- різні ліміти для гостя і авторизованого на одному й тому ж endpoint-і;
- єдина точка правди (поправив у `AppServiceProvider` — змінилося всюди);
- можна робити `Limit::perMinute(...)`, `Limit::perHour(...)`, ланцюжки лімітів (`return [Limit::perMinute(60), Limit::perDay(1000)];`), кастомний `response(...)` на 429.

4.5 Це єдиний спосіб?

Ні. На практиці трапляються три рівні:

1. **throttle:N,M у маршрутах** — швидкий мінімум, для уроків і невеликих API.
2. **RateLimiter::for(...)** + **throttle:ім'я** — стандартний production-підхід у Laravel.
3. **Зовнішній шар (Nginx, Cloudflare, API Gateway, AWS WAF)** — ліміти до того, як запит взагалі дійшов до PHP. Це не заміна throttle у Laravel, а **додатковий** контур: зовнішній шар ріже грубі атаки, Laravel — бізнес-логіку і користувацькі ліміти.

Своє middleware з нуля писати має сенс лише якщо у вас зовсім нестандартна логіка (наприклад, ліміт за конкретним тілом запиту). У 95% випадків вистачає вбудованого **throttle** + **RateLimiter::for(...)**.

4.6 Live-демо ліміту (без правки коду — навчальна демонстрація)

Покажи реакцію на перевищення ліміту (якщо тимчасово обмежити, наприклад, до **throttle:3,1**):

```
for i in {1..5}; do
  curl -s -o /dev/null -w "%{http_code}\n"
  "http://localhost/api/v1/accounts/ID/payments?per_page=10"
done
```

Очікуємо: після кількох 200 — **429 Too Many Requests**.

У заголовках:

- **X-RateLimit-Limit**
- **X-RateLimit-Remaining**
- **Retry-After** (на 429)

4.7 Що підкреслити

- Ліміти для авторизації і важких звітів завжди жорсткіші.
- Без лімітів один клієнт може «покласти» API.
- У production майже завжди використовують саме **іменовані** limiter-и через **RateLimiter::for(...)**, а не голі **throttle:60,1** у маршрутах — це і зручніше підтримувати, і точніше за бізнес-сенсом.

5) Блок D — jobs для важких операцій (перехід зі слайда 15a)

5.1 Що говорити

- Важка робота в HTTP-запиті = таймаут і поганий UX.
- Рішення: API ставить job → повертає **202 Accepted** + **task_id**.
- Воркер робить роботу у фоні, фронт опитує статус або чекає нотифікацію.

5.2 Що вже є у проекті (готово до запуску)

У проєкті під цей блок уже підготовлено все необхідне — на демо достатньо відкрити файли і запустити `curl`:

- **Контролер** `App\Http\Controllers\Api\V1\ReportsController` — метод `generateAccountStatement(Request $request)`:
 - валідує `account_id, from, to`;
 - генерує `task_id` (UUID);
 - диспатчить `ExportAccountStatementJob::dispatch(...)`;
 - повертає `202 Accepted` з `{ status, task_id, message }`.
- **Job** `App\Jobs\ExportAccountStatementJob` — implements `ShouldQueue`:
 - конструктор приймає `accountId, periodFrom, periodTo, taskId`;
 - `handle()` пише в лог `"Account statement export started"` з `task_id`,
 - знаходить акаунт, імітує важку роботу `sleep(3)`,
 - пише `"Account statement export finished"`.
- **Маршрут** `POST /api/v1/reports/account-statement` → `reports.account-statement`.
- **Воркер черги** — сервіс `queue-worker` у `docker-compose.yml` уже підхоплює джоби.

5.3 Live-демо (поетапно під час мітингу)

1. В одному терміналі — стежимо за логами в реальному часі:
 - `docker compose exec -T php sh -lc 'tail -f -n 0 storage/logs/laravel.log'`
2. В іншому — відкриваємо UI воркера/Telescope:
 - `http://localhost/telescope/jobs` (вкладка `Jobs`).
3. Робимо запит на звіт — він має відповісти **миттєво**, попри `sleep(3)` у середині джоби:

```
curl -i -X POST "http://localhost/api/v1/reports/account-statement" \
-H "Content-Type: application/json" \
-d '{"account_id":1,"from":"2026-01-01","to":"2026-01-31"}'
```

Очікуємо: `HTTP/1.1 202 Accepted`, у JSON — `task_id`.

4. У логах рівно через ~3 секунди з'явиться зв'язка рядків із цим же `task_id`:
 - `Account statement export started ... task_id=...`
 - `Account statement export finished ... task_id=...`
5. У Telescope в `Jobs` видно, що джоба оброблена воркером, не веб-процесом, і **час API-відповіді** не залежить від часу джоби.

5.4 Що підкреслити

- HTTP-відповідь повертається за десятки мілісекунд, важка робота — поза межами запиту. Це і є те, заради чого взагалі потрібні черги в API.
 - Воркер черги вже піднятий у `docker-compose (queue-worker)` — жодних ручних запусків на демо не потрібно.
 - Реальний наступний крок (поза цим уроком): окремий endpoint `GET /reports/{task_id}` зі статусом задачі та посиланням на готовий файл.
-

6) Блок Е — фінальний чек-лист production-ready API (слайд 19)

Використовуємо як «закриття» уроку на дошці:

- ☒ оптимізована БД (індекси, відсутність N+1, вимірювання через Telescope) — Модулі 9.1–9.2
 - ☒ серверний кеш (Redis) для довідників/агрегатів — 9.2
 - ☒ HTTP-кеш (**Cache-Control** / **ETag** / 304) — 9.3 Блок А
 - ☒ DTO / API Resources, контроль обсягу відповіді — 9.3 Блок В
 - ☒ пагінація всіх списків — 9.1
 - ☒ rate limiting (**throttle**) — 9.3 Блок С
 - ☒ jobs для важких операцій (202 + воркер) — 9.3 Блок D
 - ☒ тести, логи, Sentry — Модулі 7–8
-

7) Тести і логи наприкінці демо

1. Прогнати API-набір (SQLite):

- `docker compose exec -T php php artisan test tests/Feature/Api/V1`

2. Логі:

- `docker compose exec -T php sh -lc 'tail -n 100 storage/logs/laravel.log'`

3. Черги (якщо показували jobs):

- `docker compose logs queue-worker --tail=50`
-

8) Anti-fail (30 секунд до старту)

1. `docker compose exec -T php php artisan config:clear`

2. `docker compose exec -T php php artisan route:list --path=api/v1`

3. Відкрити й оновити:

- `http://localhost/telescope/requests`

4. Зробити один прогін:

- `curl -i "http://localhost/api/v1/accounts/1/payments?per_page=10"`
-

9) Карта підготовленого коду під цей урок

Що вже додано в проєкт (можна одразу запускати)

- `app/Http/Resources/PaymentFatResource.php` — «жирний» Resource (блок В).
- `app/Http/Controllers/Api/V1/AccountController::paymentsFat()` — метод, який повертає той самий список платежів через `PaymentFatResource` (блок В).
- `app/Http/Controllers/Api/V1/ReportsController — generateAccountStatement()` (блок D).
- `app/Jobs/ExportAccountStatementJob` — робочий job з логуванням і `sleep(3)` (блок D).

- В `routes/api.php` додані маршрути: `payments-cached`, `payments-fat`, `currencies`, `reports/account-statement`.

Що підготовлено як **порожні заглушки** під live-набір коду

- `AccountController::paymentsCached()` — для блока А (HTTP-кеш для персональних даних). Я вживу дописую `ETag + Cache-Control: private, max-age=15 + 304` на `If-None-Match`.
- `App\Http\Controllers\Api\V1\CurrencyController::index()` — для блока А (публічний кеш). Я вживу дописую повернення списку валют + `Cache-Control: public, max-age=3600`.

Що НЕ чіпаємо під час демо

- Наявні реалізації `AccountController::payments()`, `PaymentController`, `PaymentService`, `PaymentRepository`, `PaymentResource` і тести.
- Базовий endpoint `GET /api/v1/accounts/{account}/payments` — він залишається «як є», саме для контрасту з `payments-cached` і `payments-fat`.
- `RateLimiter / throttle` (блок С) — окремий код додавати під мітинг не потрібно: `throttle` вбудований у фреймворк, у `gunbook` показана лише конфігурація на маршрутах і через `RateLimiter::for(...)`.